# Ders 5 Genişletilmiş Notu: Decorator, Facade, Flyweight, Proxy ve Chain of Responsibility Kalıpları

## Ders 5 Genişletilmiş Notu: Decorator, Facade, Flyweight, Proxy ve Chain of Responsibility Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

## İçindekiler


- 1. Decorator Kalıbı: Amaç ve Kalıtımla Çözümün Sınırları
- 2. Decorator'ın Önerdiği Çözüm ve Kalıp Yapısı
- 3. Örnek Gerçekleme: Satış Makbuzuna Üstbilgi ve Altbilgi
- 4. Facade Kalıbı
- 5. Flyweight Kalıbı
- 6. Örnek Gerçekleme: Kimya Fabrikası
- 7. Proxy Kalıbı
- 8. Örnek Gerçekleme: İzlediğin Kadar Öde Sunucusu
- 9. Chain of Responsibility Kalıbı
- 10. Örnek Gerçekleme: Reklam Yönetimi
- Örnek Sorular
- Dersin Toparlaması


## 1. Decorator Kalıbı: Amaç ve Kalıtımla Çözümün Sınırları

Ders, yapısal (structural) kalıplardan Decorator ile başlar. Decorate "süslemek" anlamına gelir; kalıbın adı amacına uygun seçilmiştir.

### 1.1 Amaç

Kalıbın amacı, nesnelere çalışma anında ek sorumluluklar kazandırmaktır ve bunu dinamik olarak yapmaktır. Bir nesnenin görev ve sorumlulukları normalde sınıfının nasıl kodlandığına bağlıdır; ama bu kodlama zamanında belirlenir. İstenen, bir nesnenin davranışının çalışma anında değişebilmesidir.

Bunu kalıtımla yapmaya çalışmak sınıf düzeyinde, yani kod düzeyinde bir çözümdür. Kalıtımla bir özelleştirme yapıldığında yeni alt sınıfın bütün nesneleri aynı özelleşmeye sahip olur. Oysa aynı türden nesnelerin farklı farklı sorumluluklar yürütebilmesi isteniyor.

### 1.2 GoF örneği: metin alanları

Bir GUI çerçevesi (framework) hazırlanıyor. Metin alanları olsun, ama çeşitlilik de olsun: yatay kaydırma çubuğu (scroll bar) olabilen, dikey kaydırma çubuğu olabilen, bir çerçeve (border) içine alınabilen metin alanları. Birinin yalnız kaydırma çubuğu, birinin iki çubuğu, birinin yalnız çerçevesi olacak; otomatik alt satıra sarma yapılan birinde yalnız düşey çubuk olacak. Bu varyasyonların her biri için ayrı sınıf yazmak sorun olur.

İlk akla gelen kalıtımdır (ekrandan, 00:10:00). Slayttaki "Çözüm 1": TextArea sınıfının alt sınıfları ScrollTextArea, BorderTextArea ve SrcrollBorderTextArea. Hoca, hem kaydırma hem

çerçeve isteyen sınıfı tek kalıtımla TextArea'dan türetmek yerine, çoklu kalıtım olsaydı hem ScrollTextArea'dan hem BorderTextArea'dan türetilebileceğini tahtada çizdi. Slayttaki zayıf yönler:

- Scroll ve Border yetenekleri iki ayrı yerde tekrar gerçeklenmek zorundadır.
- TextArea'ya ek olarak bir de Panel sınıfına Scroll ve Border yeteneği kazandırmak için ek üç sınıf daha gerekecektir.

Hocaya göre çoklu kalıtım her dilde yoktur ve getirdiği karmaşıklık nedeniyle güncel dillerde pek tercih edilmez. Üstelik çoklu kalıtım yalnız çeşitlilik sağlar, dinamiklik sağlamaz: hazır beş metin alanı sınıfından biri seçilir, var olan bir metin alanı çalışma anında yeniden yapılandırılamaz. Ayrıca kaydırma ve çerçeve yeteneklerine ihtiyaç duyan tek bileşen metin alanı değildir; paneller de kaydırma ve çerçeve isteyebilir. Farklı bileşenlerin aynı ihtiyaçlara sahip olduğu durumda normal kalıtım da çoklu kalıtım da kurtarmaz.

### 1.3 Diamond (elmas) kalıtım problemi

Bu yapıdan doğan soruna Diamond Inheritance Problem denir. Sınıfları nokta olarak düşünüp kalıtım okları yönünde birleştirince bir elmas şekli ortaya çıkar.

Üst sınıfta, örneğin kendini çizen bir draw() metodu olsun. ScrollTextArea'da da BorderTextArea'da da bu metot override edilir. En alttaki ScrollBorderTextArea draw()'u override etmezse, kalıtımla hangi ebeveynin draw() metodunu alacağı belirsizdir. Hocanın saydığı olası kurallar:

- Kalıtım listesinde (`extends ScrollTextArea, BorderTextArea` gibi) ilk yazılan ebeveyninki geçer.
- En son yazılanınki geçer.
- C++'ta olduğu gibi böyle metotlar pure virtual tanımlanır ve en alt sınıfta mutlaka override edilmek zorunda bırakılır, yoksa derleyici hata verir.

Override edilmiş bir metodun içinden ebeveynlerin override edilmemiş sürümlerine ulaşmak gerektiğinde hangi ebeveynin kastedildiği ayrıca belirtilmelidir. Bu belirsizlikler çözülebilir ama ek karmaşıklık getirir; deneyimli dil tasarımcıları "attığımız taş ürküttüğümüz kuşa değmiyor" diye düşünmüş, güncel diller çoklu kalıtımı sınırlı desteklemiş ve arayüzler üzerinden gitmiştir. Ama arayüzlerle metot gövdesi, yani davranış aktarılamaz. Kaydırma mantığı ise metin alanlarında, panellerde ve başka yerlerde tekrar tekrar kodlanmak istenmiyor.

## 2. Decorator'ın Önerdiği Çözüm ve Kalıp Yapısı

### 2.1 Sınıfları değil, nesneleri ilişkilendirmek

Slayttaki önerilen çözüm: "Sınıfları değil de, nesneleri birbirleri ile ilişkilendirelim" (ekrandan, 00:20:00). Kaydırma mantığı bir sınıfta, çerçeve mantığı başka bir sınıfta kodlanır. Panelin kaydırmaya mı ihtiyacı var? Panel nesnesi bir kaydırma nesnesiyle ilişkilendirilir. Metnin mi ihtiyacı var? Metin nesnesi kaydırma nesnesiyle ilişkilendirilir.

Çerçevenin temel bileşenlerinin ortak bir VisualComponent üst sınıfı vardır (Draw() metodu). Farklı türden bileşenleri yönetmek için zaten böyle ortak bir sınıf gerekir. Alt sınıfları TextView ve hocanın şemaya eklediği Panel'dir.

> **Hocanın vurgusu:** GoF'un gösterimi UML standartlarına uymaz, çünkü o zaman standartlar henüz oluşmamıştı. Sınavlarda özellikle UML standardına uymanızı istiyorum: kalıtım okunu birleşim çizgisinin ortasına değil üst sınıfa dayanacak biçimde çizin (sahiplik gösterimiyle karışmasın), üye metotları public, üye alanları private olarak işaretleyin (00:14:33).

Kalıbın getirdiği yenilik Decorator sınıfıdır. Decorator, bir nesneye yeni bir sorumluluk eklemeye yarayacak sınıfları simgeleyen soyut bir üst sınıftır. Sınıf soyuttur ama bir somut metodu vardır: Draw(). Bu metot, dekoratörün ilişkili olduğu ve dekore edeceği bileşenin, yani `component` üyesinin çizilmesini sağlar: `component->Draw()`. Şemada bu ilişki parça-bütün gibi gösterilmiş olsa da aslında bire birdir: bir dekoratör nesnesi yalnız bir VisualComponent'i süsler ve ona doğrudan yeni bir yetenek kazandırır.

Decorator'ın alt sınıfları:

- ScrollDecorator: Draw(), ScrollTo(), scrollPosition. Önce `Decorator::Draw()` ile ilişkili nesnenin çizilmesine izin verir, hemen ardından kaydırma çubuklarını ekler. Çubuğa bir tıklama gelince ScrollTo ile metnin ilgili satırına kaydırır. Hoca, slaytta yatay-dikey ayrımı yapılmadığını, yapılsaydı HorizontalScrollDecorator ve VerticalScrollDecorator diye ayrı sınıflar gerekeceğini söyledi.
- BorderDecorator: Draw(), DrawBorder(), borderWidth. Draw() notta yazdığı gibi `Decorator::Draw(); DrawBorder();` yapar: bileşen çizildikten sonra ek olarak çerçeve çizilir. Çerçeveyi çizeceği bileşene component üyesi üzerinden referansı olduğu için koordinatları alabilir; kaç piksel kalınlıkta çizeceği gibi ek bilgiler için ek alanları olur.

Nesne düzeyinde bakıldığında, bir metin alanının hem kaydırması hem çerçevesi olacaksa bir BorderDecorator ve bir ScrollDecorator oluşturulur. aScrollDecorator, aTextView'u dekore eder ve ona kaydırma yeteneği kazandırır; ikisi artık tek bir VisualComponent gibi davranır. Üstüne aBorderDecorator konur ve daha büyük üçüncü bir bileşen ortaya çıkar. İstemci bu bileşenle en dıştaki dekoratör üzerinden etkileşir; draw dendiğinde hepsi çizilir ve ek işler gerektiği gibi yürütülür.

### 2.2 Kalıp yapısı

Slayttaki kalıp yapısı (ekrandan, 00:22:00):

```
Component (soyut)            Operation()
  ConcreteComponent          Operation()
  Decorator (soyut)          Operation()  -> component->Operation()
    [component : Component]  (Decorator'dan Component'e bire bir sahiplik)
    ConcreteDecoratorA       Operation(), addedState
    ConcreteDecoratorB       Operation(), AddedBehavior()
                             Operation() -> Decorator::Operation(); AddedBehavior();
```

Kalıbın getirdiği kısım Decorator ve alt sınıflarıdır. Decorator, hangi bileşenle ilişkilendirildiyse işi ona yaptıracak biçimde kodlanır. Somut alt sınıflarda önce asıl operasyon yürütülür, sonra ek davranış yürütülür. Hoca, ek davranışın öne de alınabileceğini söyledi: önce ek iş sonra normal iş, ya da tersi; hangisi probleme uygunsa.

İnsan kaynakları örneği: bazı işlerden önce mutlaka iş sağlığı ve güvenliği izni alınmalıdır, bazı işlerden sonra iş tamamlama tutanağı birine teslim edilmelidir, bazılarında ikisi de gerekir. Her işte güvenlik ya da tutanak gerekmez. Bu durumda bir dekoratör önce ek işi, bir başkası sonra ek işi yapar. Koşullu da olabilir: doktor hastaysa hasta kabul etmez; dekoratör önce bir if ile denetler, koşul doğruysa operasyonu yaptırır, değilse yaptırmaz. Somut dekoratörün operasyonunda bu esneklik vardır.

> **Hocanın vurgusu:** Decorator'ın operasyonu, dekore edilen bileşenin operasyonunun unutulmaması içindir. Somut alt sınıfın operasyonunda bunu uygun bir yerde çağırmayı unutmamak gerekir.

### 2.3 Uygulanabileceği anlar

- Tek tek nesnelere dinamik olarak, yani çalışma anında yeni sorumluluklar eklemek isteniyorsa.
- Bu sorumlulukları şeffaf olarak, aynı türden diğer nesneleri etkilemeden eklemek isteniyorsa. Hasta doktorun rolü askıya alınır, ama diğer doktorların görevleri devam eder.
- Bazı sorumlulukların iptal edilebileceği ya da askıya alınabileceği durumlarda.
- Kalıtımdan uzak durulmak istenen bazı senaryolarda.

### 2.4 Java'daki stream sınıfları

Hocanın aklına gelen tam oturan bir örnek, Java'nın stream tabanlı giriş-çıkış sınıflarıdır; bunlar Decorator kalıbıyla gerçeklenmiştir. Temel input stream sınıfı yalnız bayt okur ve yazar, bununla çok yararlı bir iş yapılamaz. Metin girişi için yararlı metotlar (satır satır, karakter karakter okuma) içeren bir stream sınıfı kurulurken ona bir input stream verilir. Komple nesneler okunacaksa ObjectInputStream'in kurucusuna bir FileInputStream verilir; böylece ObjectInputStream, FileInputStream'i dekore etmiş olur ve readObject ile nesneler okunabilir. Veri

sıkıştırılmış bir dosyadan okunacaksa, kurucusuna dosya akışı verilen bir zip input stream oluşturulur, o da nesne ya da metin okuyan akışa verilir. Hoca sınıf adlarının tam olarak böyle olmayabileceğini, ama mantığın bu olduğunu söyledi: çekirdek bir yetenek yeni yeteneklerle istenen biçimde geliştirilerek kullanılır.

> **Ek bilgi (derste anlatılmadı):** Java'da bu yapının tepesinde InputStream vardır; FilterInputStream dekoratör üst sınıfı rolündedir ve BufferedInputStream, DataInputStream gibi sınıflar ondan türer. ObjectInputStream ve java.util.zip altındaki ZipInputStream, GZIPInputStream de kurucularında başka bir InputStream alır. Metin okuma tarafında ise büyük olasılıkla Reader ailesi (BufferedReader, InputStreamReader) kastedilmiştir.

### 2.5 Zayıf yönler

Yeni yeteneğin eklenmesinin üç olası biçimi vardır: eski sorumluluktan önce bir iş yapmak, sonra bir iş yapmak ya da eski sorumluluğu tamamen iptal etmek. Bileşenler birbirini dekore ettiği için bir sorumluluk tamamen iptal edilirse, ilişkili olduğu diğer bileşenlerin kodunun çağrılmamasına, yanlış davranış ve hesaplamalara yol açılabilir. Sorumluluk askıya alınırken daha dikkatli olmak gerekir.

İkinci nokta: nesneye en içteki dekore edilmiş nesne üzerinden değil, en son kim dekore ettiyse onun üzerinden erişilir. Aradaki bir nesneye erişmek için ona ayrı bir işaretçi tutmak gerekir; ama bu da pek doğru olmayabilir. Madem bir ek sorumluluklar zinciri kuruldu, onu koparmaktansa ihtiyaç duyulan ara halkaya sahip yeni bir zincir oluşturmak daha doğrudur. Hoca, kalıbın tamamen davranış değiştirmek için değil, yeni görev ve sorumluluk eklemek ya da askıya almak için ideal olduğunu tekrarladı.

## 3. Örnek Gerçekleme: Satış Makbuzuna Üstbilgi ve Altbilgi

> **Ek bilgi (derste anlatılmadı):** SalesTicket, Header ve Footer sınıflarıyla kurulan bu makbuz örneği büyük olasılıkla Shalloway ve Trott'un "Design Patterns Explained" kitabındaki Decorator örneğidir.

### 3.1 Problem ve ilk çözüm

Bir bilgi sistemi var; e-ticaret de olabilir, stok yönetimi de. Satış emirleri geliyor ve bunların makbuzlarının basılması gerekiyor. Mevcut programın denetleyicisi TaskController'dır. Slayttaki "Çözüm 1: Kullanım ilişkisi ile" (ekrandan, 00:30:00):

```
TaskController ---> SalesOrder (+process(), +prtTicket())
SalesOrder     ---> SalesTicket (+prtTicket())
SalesTicket    ---> Header (+prtHeader())
SalesTicket    ---> Footer (+prtFooter())

prtTicket:
  if want header info, call Header's prtHeader method
  call SalesTicket's prtTicket method
  if want footer info, call Footer's prtFooter method
```

SalesOrder'ın ayrı bir prtTicket metodu olması şart değildir; makbuz basımı process metodunun içine de gömülebilirdi. Her durumda SalesTicket ile bir ilişki kurulur ve makbuz onun prtTicket metodu üzerinden basılır.

Sonra yeni bir istek gelir: makbuzlara üstbilgi ve altbilgi basılmalıdır. Bir Header ve bir Footer sınıfı oluşturulur; SalesTicket, başlık gerekiyorsa prtHeader ile başlığı yazar, makbuzu basar, altbilgi gerekiyorsa if ile denetleyip prtFooter'ı çağırır. Slayttaki soru: farklı üstbilgi ve altbilgi türleri söz konusu ise ne olacak?

### 3.2 İkinci adım: kalıtımla türler

Farklı siparişlerde farklı türde üstbilgi ve altbilgi istenir. Header1, Header2, Footer1, Footer2 diye devam edilirse ve bu iş if-else ile yapılırsa uzun bir zincir çıkar: "şu ise header 1, değilse

şu ise header 2, değilse şu ise header 3…”.

> **Hocanın vurgusu:** Bu şekilde kod yazdığınız zaman aklınızda bulunsun: mutlaka bir yanlış yapıyoruz, daha iyi bir tasarım yapabiliriz. Bunu kalıtım ve çok biçimlilik kullanarak toparlamalıyız (00:35:06).

İki ayrı kalıtım hiyerarşisi kurulur: soyut Header ve soyut Footer, bunların somut alt sınıfları Header1, Header2 vb. SalesTicket'a hangi tür header ve footer gerekiyorsa o verilir; yeni bir tür gerektiğinde yalnız bir kardeş sınıf eklenir.

Ama sonra “iki header birden basılsın” ya da “iki footer birden basılsın” isteği gelir. Footer1 ve Footer2'nin birlikte olduğu yeni bir alt sınıf yazmak gibi, her kombinasyon için ayrı alt sınıflar gerekir; bu yoldan da gidilemez.

### 3.3 Decorator çözümü

Tasarım kalıpları bilinirse burada akla Decorator gelir: “Benden süslü makbuz istiyorsunuz; istediğiniz kadar süsleyebileceğiniz makbuzları Decorator ile halledeceğim.” Bir durumda header1, header2, makbuz, footer2; başka bir durumda header1, makbuz, footer1, footer2 gibi aklınıza gelen her düzen kolayca kurulur.

Sınıflar (paket `dp.decorator.example`):

- Component: soyut sınıf, `prtTicket()` soyut metodu. Yalnız ortak bir erişim altyapısı sağlar.
- SalesTicket extends Component: makbuzu basar.
- TicketDecorator: soyut, Component'ten türer, dekore edilen bileşen için `myComponent` üyesi tutar.
- Header1, Header2, Footer1, Footer2: TicketDecorator'dan türer.
- Factory: farklı dekorasyon kombinasyonlarını kurar.
- Client: Factory'den bileşeni alıp prtTicket'ı çağırır.

İstemci Component'lerle konuşur; çoğu zaman bir SalesTicket ile konuşuyor gibidir, ama nasıl dekore edilmesi gerekiyorsa öyle dekore edilmiş bir bileşenle konuşur.

TicketDecorator'ın kodu (ekrandan, 00:43:40):

```
package dp.decorator.example;

public abstract class TicketDecorator extends Component {
    private Component myComponent;
    public TicketDecorator (Component myComponent) {
        this.myComponent= myComponent;
    }
    public void prtTicket () {
        if (myComponent != null) myComponent.prtTicket();
    }
}
```

Hoca, dekore edilen nesnenin null olmadığından emin olunması gerektiğini ve bunun kalıpta söylenene bir ek olduğunu belirtti. Dekore edilecek nesne verilmemişse ve yeterli bilgi varsa o nesne ilklendirilebilir de.

Header1'in kodu (ekrandan, 00:44:30):

```
package dp.decorator.example;

public class Header1 extends TicketDecorator {
    public Header1 (Component myComponent) {
        super(myComponent);
    }
    public void prtTicket () {
        // place printing header 1 code here
        System.out.println(getClass().getName());
        super.prtTicket();
```

```
    }
}
```

Header'lar önce kendi başlığını yazar, sonra üst sınıfın prtTicket metodunu çağırarak dekore ettiği bileşenin basılmasını sağlar. Header2 tamamen aynıdır. Footer'lar ise tersini yapar: önce `super.prtTicket()` ile içteki basımı yaptırır, sonra kendi altbilgisini yazar. Kurucu, hangi bileşenle ilişkili olacağını üst sınıfın kurucusuna iletir ve ilişki orada kurulur.

### 3.4 Java Reflection ile sınıf adını yazmak

Makbuz basılınca sipariş numarası gibi gerçek bilgiler yazmak yerine, hoca hangi sınıftan hangi metodun çağrıldığını görmek için sınıf adını konsola yazdırdı. Bunun için Java Reflection (öz inceleme) kullanılır. Object sınıfından gelen getClass() metodu, nesnenin ait olduğu sınıfı simgeleyen bir Class nesnesine işaretçi döndürür. Bu Class nesnesiyle sınıf hakkında birçok bilgiye erişilir: getName() ile sınıfın adı, sınıfın üye alanlarının ve üye metotlarının listesi; belirli bir nesnenin belirli bir metodunu istenen parametrelerle çalıştırmak gibi meta düzeyde işlemler de yapılabilir.

Özetle, bir nesnenin hangi sınıfa ait olduğunu tam adıyla öğrenmek için `nesne.getClass().getName()` çağrılır. Bir SalesTicket nesnesinin prtTicket metodu `dp.decorator.example.SalesTicket` yazar, çünkü tam nitelikli sınıf adı (fully qualified class name) budur. Böylece çağrı zincirinde kimin kimi çağırdığı görülebilir.

> **Ek bilgi (derste anlatılmadı):** Java'da bir metodu yansıma yoluyla çalıştırmak için `Class.getMethod(...)` ile alınan `java.lang.reflect.Method` nesnesinin `invoke` metodu kullanılır; hocanın "execute" diye andığı işlem büyük olasılıkla budur.

### 3.5 Fabrika ve çalıştırma

Hocanın aklına Factory Method da gelmiş: farklı header-footer kombinasyonları için bir fabrika sınıfı olsun. Farklı kombinasyonlar için farklı fabrika sınıfları da olabilir, ama burada bir tane yetiyor. Hoca, bu kodun istemcinin içine de konabileceğini, basitlik için böyle ayırdığını söyledi.

Fabrika, aynı nesne referansını yeniden kullanarak zinciri kurar: önce bir SalesTicket oluşturulur, sonra onu bir dekoratör sarar, yeni bileşeni bir başkası sarar. Ekrandaki son hali (ekrandan, 00:50:00):

```
package dp.decorator.example;

public class Factory {
    public Component getComponent () {
        Component myComponent;
        myComponent = new SalesTicket();
        myComponent = new Footer1(myComponent);
        myComponent = new Footer2(myComponent);
        myComponent = new Header2(myComponent);
        myComponent = new Header1(myComponent);
        return myComponent;
        /*
        return new Header1( new Header2( new Footer2(
                new Footer1( new SalesTicket() ) ) ) );
        */
    }
}
```

Bu düzenin konsol çıktısı:

```
dp.decorator.example.Header1
dp.decorator.example.Header2
dp.decorator.example.SalesTicket
dp.decorator.example.Footer1
dp.decorator.example.Footer2
```

Adım adım (ara adım eklendi): en dıştaki Header1'in prtTicket'ı çağrılır, "Header1" yazar ve içindeki Header2'ye iletir. Header2 "Header2" yazar ve Footer2'ye iletir. Footer2 önce kendi işini değil içtekinin işini ister ve Footer1'e iletir. Footer1 de önce içtekini, yani SalesTicket'ı çalıştırır: "SalesTicket" yazılır. Sonra Footer1 kendi satırını, en son Footer2 kendi satırını yazar.

Hoca derste sıraları değiştirerek iki gözlem yaptı:

- Header'ların ilişkilendirme sırası değiştirilince (önce Header1 sonra Header2 ile sarmak) en dışta Header2 kalır ve çıktı Header2, Header1 olur (00:47:34).
- Footer'lar header'lardan önce ya da sonra ilişkilendirilse de footer'lar her zaman makbuzdan sonra basılır, çünkü footer önce içteki işi yaptırıp sonra kendi işini yapacak biçimde kodlanmıştır. Önce Footer1 sonra Footer2 ile sarılınca çıktı Footer1, Footer2 olur.

Hocanın vardığı sonuç: biri her zaman önce, biri her zaman sonra çalışacak biçimde kodlandığı için header-footer ilişkilendirme sırası çıktıyı etkilemez. Ama aynı amaca hizmet eden dekoratörlerin (iki header ya da iki footer) kendi aralarındaki ilişkilendirme sırası çıktı sırasını etkiler. Doğru dekorasyonu ilk başta çıkaramazsanız bile yazıp çalıştırınca görürsünüz; yanlışsa sırayı değiştirirsiniz.

Hoca soru sorulup sorulmadığını kontrol etti; sohbette ileride görülecek Strategy kalıbıyla ilgili bir not vardı, henüz görülmediği için üzerinde durulmadı.

## 4. Facade Kalıbı

Facade, İtalyanca kökenli bir kelimedir. Kalıbın amacı, çeşitli alt sistemlerde bulunan farklı arayüzleri tek ve daha üst düzey bir arayüzde toplamaktır; böylece istemciler karmaşık sistemin ayrıntılarını bilmek zorunda kalmadan tek bir erişim noktası üzerinden işlerini yapabilir.

Elimizde hazır bir alt sistem olsun: bir dolu sınıf, ilişki, kalıtım. İstemcilerin bazı sınıfların bazı metotlarını çağırması gerekiyor. Facade olmadan istemciler bu metotların hangi sınıfta, hatta hangi pakette olduğunu bilmek zorundadır. İp yumağı gibi karmaşık bir yapının içinden belirli sınıfların belirli metotlarını bilmek yerine bir erişim noktası verilir; istemci isteğini ona iletir, metodun hangi sınıfta olduğunu o bilir.

Adın kaynağı binaların ön yüz (dış cephe) kaplamasıdır: kaplamayı değiştirerek binanın dış görünüşünü değiştirebilirsiniz. Hocaya göre kalıbı daha iyi anlatan benzetme şudur: karmaşık bir bürokratik sistem var ve vatandaşlar işleri düştüğünde oda oda, birim birim dolaşmıyor; kapıda bir danışman var. Ona işlerini söylüyorlar, o işin nerede yapılacağını ve hangi belgelerin lazım olduğunu biliyor, işlemlerin yapılmasını sağlıyor ve "iki saat sonra gelin" diyor. Facade, arkadaki karmaşık sistemi örten bir danışmandır.

Kalıp çok basit olduğu için hoca örnek gerçekleme yapmadı. Uygulanabileceği anlar:

- Karmaşık bir sisteme basit bir arayüz sağlanmak isteniyorsa.
- Sistem, kullanıcının etkileşim düzeyine göre katmanlara ayrılmak isteniyorsa.
- Bağlaşım (coupling) azaltılmak isteniyorsa: farklı istemciler farklı sınıflarla bağlaşım içinde olacakken artık yalnız Facade ile bağlaşımdadır, Facade da diğer sınıflarla. Facade olmasaydı şemada istemcilerden ihtiyaç duydukları her sınıfa ayrı oklar çizilecekti.

## 5. Flyweight Kalıbı

Aradan sonra yapısal kalıplardan Flyweight işlendi.

### 5.1 Ad ve amaç

Flyweight adı boks gibi sporlardaki sıklet sınıflarından gelir: sinek sıklet, hafif sıklet, orta sıklet, ağır sıklet. Flyweight en hafif klasmandır, Türkçede tüy sıklet ya da sinek sıklet diye geçer. Kalıbın temel amacı, paylaşım yoluyla çok sayıda basit nesnenin kullanımını desteklemektir. Nesneler basit ve çok sayıda olduğu için bu ad verilmiştir.

### 5.2 GoF örneği: metin editörü

Word gibi bir metin editörü yazılıyor. Karakterler yazı tipi ve diğer biçim bilgilerine göre (italik mi, koyu mu) en iyi biçimde gösterilmek, yani mümkün olan en iyi render işlemi yapılmak isteniyor. Bu amaçla harfler grafik nesneleri olarak ele alınıp ekrana çizilecek. Problemi karıştırmamak için sabit genişlikli yazı tipi ve daktilo gibi sayfada belirli sayıda satır ve sütun olduğu, her kutucuğa bir harf geldiği düşünülür.

Sorun şudur: editöre yazılan her harf, her kutuda ayrı bir nesne olarak ele alınırsa çok fazla bellek harcanır, dosyada da çok yer kaplar. Hoca slayttaki metinde büyük A harfinin yalnız başlıkta beş yerde geçtiğini gösterdi. Hocanın varsayımsal sayılarıyla, her harf nesnesi 100 kilobayt olsa her A için ayrı nesne çok yer tutar; ders için hazırladığı üç sayfalık bir öneri formunda bile beş bin kadar harf vardı.

Çözüm: sayfanın her satır-sütun gözünde ayrı bir harf nesnesi oluşturmak yerine, bütün küçük ve büyük harfler bellekte ortak paylaşılan bir havuzda tutulur ve ilgili satır-sütun konumlarında havuzdaki nesnelere referans verilir. Dosya kaydedilirken önce havuz yazılır; büyük A görülen yerde havuzdaki birinci nesneye, B görülen yerde ikinci nesneye referans tutulur. Harf nesnesinin kendisi onlarca kilobayt olabilirken ona işaret eden bir tamsayı 4 bayttır; böylece epey tasarruf edilir.

Paylaşılan nesnelere flyweight denir. Bu nesnelerin “sinek sıklet” sayılmasının sebebi, havuz işaretçileri üzerinden kullanılmalarıdır; yoksa nesnelerin kendisi epey ağır olabilir ve en çok tasarruf da bu durumda olur.

Slayttaki örnek yapıda satırlar bir listede tutulur ve belirli bir satırın belirli bir sütunundaki harf basılır; bu yüzden satır (Row) ve sütun (Column) diye iki sınıf, bir de karakter (Character) sınıfı vardır. Hepsi Glyph, yani sembol sınıfından türer. Satır çizileceği zaman çocukları olan sütun nesnelerini çizer, sütun da ilişkili olduğu karakter nesnesini çizer.

### 5.3 Kalıp yapısı

Ortak havuzu idare edecek bir sınıf gerekir; kalıp buna FlyweightFactory der. Slayttaki yapı (ekrandan, 01:20:00):

```
FlyweightFactory   GetFlyweight(key)          -- flyweights --> Flyweight (soyut)
  GetFlyweight:
    if (flyweight[key] exists) {
        return existing flyweight;
    } else {
        create new flyweight;
        add it to pool of flyweights;
        return the new flyweight;
    }
Flyweight (soyut)             Operation(extrinsicState)
  ConcreteFlyweight           Operation(extrinsicState), intrinsicState
  UnsharedConcreteFlyweight   Operation(extrinsicState), allState
Client --> FlyweightFactory, ConcreteFlyweight, UnsharedConcreteFlyweight
```

Fabrikaya bir anahtar verilerek ilgili flyweight nesnesine erişilir. Metin örneğinde anahtar küçük “a” karakteri ise fabrika küçük a harfinin bitmap resminin işaretçisini verir. Var olmayan bir anahtar istenirse ya null döndürülür ya da ilgili nesne oluşturulup havuza eklenir ve döndürülür; probleme hangisi uygunsa.

Şemadaki kalıtımın nedeni: bazı flyweight’lerin paylaşılması gerekmeyebilir ya da uygun olmayabilir. Normal çalışmada bütün küçük a harfleri bellekte aynı konumdaki, aynı kimlikli nesneye işaret eder. Bazı problemlerde ise nesnelerin dış görünüşleri ve durum bilgileri aynı olsa da farklı yerlerde saklanan farklı nesneler olduğu bilgisinin, yani nesne kimliğinin kaybolmaması gerekir. Böyle durumlarda ortak Flyweight üst sınıfından paylaşılan (ConcreteFlyweight) ve paylaşılmayan (UnsharedConcreteFlyweight) nesneler ayrı alt sınıflar olarak üretilir. Böyle bir ihtiyaç yoksa üst sınıfa da paylaşılmayan alt sınıfa da gerek yoktur; fabrika doğrudan somut flyweight’e erişir.

### 5.4 Intrinsic ve extrinsic durum

Bir nesnenin flyweight olarak kullanılabilmesi için durum bilgisinin iki parçaya ayrılabilmesi gerekir: extrinsic (dışsal) ve intrinsic (içsel) durum. Dışsal bilgi, bir işlemin yapılması için dışarıdan hesaplanıp nesneye verilen bilgidir. İçsel bilgi, nesnenin kendisinde yer alan ve değişmeyecek bilgidir.

Metin editöründe somut flyweight'lerin içsel durumu harflerin bitmap resimleri, dışsal durumu ise belirli bir harfin hangi satır ve sütunda yer aldığıdır. Dışarıdan "şu satır ve sütuna şu harf" denir, harf nesnesi de kendi iç bilgisini oraya dökerek kendini yazar. İçsel durum ne kadar fazlaysa ortak nesneyle o kadar çok tasarruf yapılır.

Hoca bir soru üzerine şunu ekledi: içsel bilgi immutable (değişmez), dışsal bilgi mutable (değişebilir) diye de söylenebilir. Bir harfin hangi satır ve sütunda yer aldığı değişebilir, ama nasıl render edildiği, şekli değişmez. İçsel kısmı oluşturan alanlar isterseniz final olarak tanımlanabilir.

> **Hocanın vurgusu:** Singleton'ı görmüştük: bir nesnenin tek örneğinin olması gereken durumlar. Flyweight'ler bir bakıma Singleton'dır; A harfinden tek bir tane vardır. Ama Singleton kalıbını ayrıca kurmaya gerek yoktur, şart değildir (01:23:24).

Sohbetten Memento kalıbıyla ilgili bir soru geldi; hoca bunu Memento kalıbı işlenirken konuşacaklarını söyledi.

### 5.5 Uygulanabileceği anlar

Hocaya göre aşağıdaki koşulların hepsi doğruysa kalıbı kullanmak mantıklıdır, aksi halde kalıp avantajını kaybeder:

- Uygulama çok sayıda nesne kullanıyorsa.
- Nesnelerin çok sayıda olması nedeniyle saklama masrafı yüksekse.
- Nesnelerin durum bilgisi içsel ve dışsal olarak ikiye ayrılabiliyorsa.
- Dışsal durum ayrıldıktan sonra çok sayıda nesnenin yerini az sayıda paylaşılan nesne alabiliyorsa.
- Uygulama nesne kimliğine bağlı değilse.

Son koşul seçimlik sayılabilir: nesne kimliğine bağlılık varsa kalıp yine kullanılabilir, ama UnsharedConcreteFlyweight'ler oluşturmak gerekir; uygulama daha ayrıntılı olur ve elde edilen tasarruf azalır. Diğer koşullar ise sağlanmalıdır. Durum bilgisi içsel ve dışsal olarak ayrılamıyorsa kalıp zaten uygulanamaz. Nesne sayısı çok değilse kalıbı uygulamak "attığınız taş ürküttüğünüz kuşa değmez".

## 6. Örnek Gerçekleme: Kimya Fabrikası

### 6.1 Problem

Bir kimya fabrikası için üretim bilgi sistemi ya da katalog hazırlanıyor. Yalnız saf kimyasal maddeler değil, birkaç maddenin bir araya getirildiği karışımlar da satılıyor. Bir kimyasal madde birden fazla karışımda yer alabilir.

### 6.2 İlk çözüm

Slayttaki ilk çözüm (ekrandan, 01:30:00):

```
KimyasalMadde
  - gram : double
  - isim : String
  - simge : String
  - atomAgirligi : double
  + Kimyasal( isim : String, simge : String, atomAg : double )
  + setGram( double )
  + getGram( ) : double
  + getIsim( ) : double
  + getAtomAgirligi( ) : double
```

```
  + getMol( ) : double          -- return gram/atomAgirligi;

Karışım  (KimyasalMadde'leri bir araya getirir)
  - isim : String
  - maddeler : LinkedList<KimyasalMadde>
  + setIsim( String )
  + getIsim( ) : String
  + maddeEkle( KimyasalMadde )
  + tarifEt( ) : String
```

Ders sonrası not: Slaytta kurucu `Kimyasal(...)` diye ve getIsim()'in dönüş türü double yazılmış; sınıf adına ve alanın türüne göre doğrusu büyük olasılıkla `KimyasalMadde(...)` ve `getIsim() : String` olmalıdır. Hoca derste şemadaki `LinkedList<KimyasalMade>` yazım hatasını fark edip slayta "madde olacak" açıklamasını ekledi (ekrandan, 01:40:00).

KimyasalMadde'de maddenin kaç gramlık paketler halinde satıldığı (gram), adı, periyodik tablodaki simgesi ve atom ağırlığı tutulur. Gram için set metodu vardır, diğerleri için yoktur: karbon her zaman karbondur, simgesi C'dir ve literatürde yerleşmiştir, atom ağırlığı değişirse başka bir madde olur. Kimyacılar ağırlığı mol cinsinden de ister; mol, gram miktarının atom ağırlığına bölümüdür. Karışımın bir adı vardır; içindeki maddeler dizi yerine Java'nın jenerik sınıf özelliğiyle bir LinkedList'te tutulur. tarifEt metodu karışımda hangi maddeden kaç gram bulunduğunu anlatan bir String döndürür.

Hoca, kaynak kodun basit olduğu için notlarda verilmediğini, sıkıştırılmış dosyadaki dp.flyweight.solution1 paketlerinde olduğunu, a çözümünün doğru olmayan asıl çözüm, b ve c çözümlerinin de doğru olmayan alt maddelerle ilgili çözümler olduğunu slayttan okudu. Slayttaki örnek: 100 gr barutta 75 gr potasyum nitrat, 15 gr karbon, 10 gr kükürt bulunur.

### 6.3 Kara barut ve temiz barut

Ana programda iki karışım oluşturulur. Kara barut: Sodyum Nitrat (NaNO3, 84.9947) 75 gram, Karbon (C, 12.0107) 15 gram, Kükürt (S, 32.066) 10 gram. Temiz barutta sodyum nitrat yerine potasyum nitrat (KNO3) kullanılır, diğer bileşenler aynıdır (ekrandan, 02:00:00).

Çözüm a'da her karışıma eklenen kimyasal madde ayrı bir nesnedir. Karbon ve kükürt iki karışımda da vardır; durum bilgileri aynı olsa da bellekte farklı yerlerdeki farklı nesnelerdir. Çok sayıda karışım düşünüldüğünde aynı türden nesnenin birçok karışımda tekrar tekrar yer alması depolama maliyetini artırır.

Hoca ilk başta bu durumun bir avantajı olduğunu da gösterdi: karbon iki karışımda farklı gramajlarla (birinde 15, diğerinde 14 gram) kullanılmak istenirse çözüm a bunu destekler, çünkü iki karbon nesnesi ayrıdır; birinin 14 gram olması diğerinin 15 gram olmasını bozmaz. Hoca toplamların 100 gram etmesi için temiz barutta karbonu 14, kükürdü 11 gram yaptı.

> **Ders sonrası not:** Kodda (ekrandan, 02:00:00) Potasyum Nitrat (KNO3) için de sodyum nitratın değeri olan 84.9947 verilmişti. KNO3'ün mol kütlesi büyük olasılıkla yaklaşık 101.10 g/mol'dür (K 39.0983, N 14.0067, O 15.9994 ile python'da $39.0983 + 14.0067 + 3 \times 15.9994 = 101.1032$; NaNO3 için aynı hesap 84.9947 verir). Hoca aradan sonra atom ağırlıklarını düzelttiğini söyledi (02:09:50).

### 6.4 Çözüm b: tek ortak nesne ve yanlış sonuç

Tasarruf için karbona ortak tek bir nesne kullanılırsa (çözüm b), kara barutta 15 gram olarak ayarlanan karbon nesnesi temiz barut için `c.setGram(14)` yapılınca kara baruttaki gramaj da 14 olur. Ekranda iki karışımda da "14.0gr Karbon" yazar ve `c.gr=14.0` çıktısı görülür (ekrandan, 02:00:00). Aynı kavramın farklı karışımlarda bulunması sorunu budur: son atanan değer bütün karışımlarda geçerli olur.

Hocanın değerlendirmesi: a ve b çözümlerinin karşılıklı avantaj ve dezavantajları vardır. Çözüm a ideal değildir, nesnelerin aynı bilgisi birçok yerde tekrarlanır ve bellek harcaması çoktur; ama en azından doğru sonuç üretir. Çözüm b bellek tasarrufu yaparken yanlış sonuç verir. Hoca solution1b ile solution1c'yi karşılaştırınca aralarında yalnız gramın bir metotla atanması

gibi küçük bir fark olduğunu, gram hâlâ aynı madde nesnesinde tutulduğu için bunun ayrı bir varyasyon sayılmasının pek anlamı olmadığını söyledi.

### 6.5 Daha doğru modelleme: Bileşen sınıfı

Karbon grafit gibi pek çok bileşiğin içinde farklı gramajlarla yer alır. Bunu bellek tasarrufu yaparak nasıl modelleriz? Temel ilke hatırlanır: neyin değiştiğini bul ve onu sarmala. Gram bilgisi madde sınıfıyla çok ilgili değildir; maddeler farklı karışımlarda farklı gramajlarla yer alır. Bu yüzden gram KimyasalMadde'den çıkarılır ve gram ile maddeyi tutan yeni bir Bileşen sınıfına konur. Hoca bunu ileride görülecek Extract Field ve hatta Extract Class refactoring'ine bağladı.

Slayttaki tasarım (dp.flyweight.solution2, ekrandan 01:50:00):

```
Bileşen
  - gram : double
  - madde : KimyasalMadde          (Bileşen 1 --> 1 KimyasalMadde)
  + Bilesen( double, KimyasalMadde )
  + getGram( ) : double
  + getMol( ) : double
  + getMaddeAdi( ) : String

Karışım  (Bileşen'leri bir araya getirir)
  - isim : String
  - bilesenler : LinkedList<Bileşen>
  + setIsim( String )
  + getIsim( ) : String
  + bileşenEkle( madde : KimyasalMadde, gram : double )
      veya: bileşenEkle( Bileşen )
  + tarifEt( ) : String

KimyasalMadde
  - isim : String
  - simge : String
  - atomAgirligi : double
  + Kimyasal( isim : String, simge : String, atomAg : double )
  + getIsim( ) : double
  + getAtomAgirligi( ) : double
  + getMol( gram : double ) : double
```

Bir bileşen tek bir kimyasal madde içerir ve artık farklı karışımlarda farklı gramajlarla yer alabilir. Karışıma bileşen eklerken yalnız madde vermek yetmez, kaç gram kullanılacağı da verilmelidir. İki seçenek vardır: `bileşenEkle(madde, gram)` ile madde ve gram ayrı ayrı verilip metot içinde Bileşen nesnesi oluşturulur ya da Bileşen dışarıda oluşturulup `bileşenEkle(Bileşen)` ile verilir. Hoca, geriye doğru uyumluluk istenirse metot adının maddeEkle olarak tutulabileceğini, ama bunun bir tasarım oluşturma süreci olduğunu söyledi.

Slayttaki sonuç: bu noktada gram, dışsal (extrinsic) bilgi olarak dikkati çeker; çok sayıda madde nesnesi olacağı da düşünülünce Flyweight kalıbına yönelinir. Hoca, temel işin aslında burada gerçekleştiğini ve burada durulabileceğini de belirtti: bir karbon nesnesi oluşturulduktan sonra farklı karışımlara farklı gramajlarla eklenebilir hale gelindi.

### 6.6 Flyweight çözümü

Flyweight kullanılırsa madde fabrikası, üretilen maddelerin dökümünü almak için de yararlı bir sınıf olur. Anahtar simgedir, gram dışsal bilgidir. Slayttaki tasarım (dp.flyweight.solution3, ekrandan 01:52:00): Bileşen ve KimyasalMadde aynı kalır; Karışım'ın ekleme metodu `bileşenEkle( simge : String, gram : double )` olur ve yeni MaddeFabrikasi sınıfını kullanır:

```
MaddeFabrikasi
  - maddeler : HashMap<String, KimyasalMadde> { static }
  + maddeBul( simge : String ) : KimyasalMadde { static }
```

Fabrikada yalnız maddeBul değil, maddeleri listeleyen bir metot da olabilir. Hem havuz hem maddeBul static yapılmıştır, yani biraz Singleton gibi. Ama öyle olmak zorunda değildir; maddeler farklı gruplar halinde tutulmak istenirse fabrikanın birden fazla örneği de olabilir. Hocaya göre farklı fabrikalarda aynı maddenin kopyaları olabileceği için static yapmak en fazla bellek tasarrufunu sağlar.

Flyweight çözümünün ana programı (ekrandan, 02:10:00):

```
package dp.flyweight.solution3;
public class MainApp {
    public static void main(String[] args) {
        Karisim karaBarut = new Karisim();
        karaBarut.setIsim("Kara barut");
        karaBarut.maddeEkle("NaNO3", 75);
        karaBarut.maddeEkle("C", 15);
        karaBarut.maddeEkle("S", 10);
        Karisim temizBarut = new Karisim();
        temizBarut.setIsim("Temiz barut");
        temizBarut.maddeEkle("KNO3", 75);
        temizBarut.maddeEkle("C", 14);
        temizBarut.maddeEkle("S", 11);
        System.out.println( karaBarut.tarifEt() );
        System.out.println( temizBarut.tarifEt() );
    }
}
```

Slayttaki şemada metodun adı bileşenEkle, kodda maddeEkle olarak geçmektedir. Kodda istemci yalnız simge ve gram verir; madde nesnesini fabrikadan karışım alır.

### 6.7 Sohbetten gelen soru

**Soru:** Her nesne için gerekmeyen ya da kullanılmayan alanların olması bir olumsuzluk oluşturmaz mı? Gram final ya da static olsaydı nasıl olurdu?

**Cevap:** Aynı madde farklı bileşenlerde farklı gramlarla kullanıldığı için gram dışsal bilgidir, zaman içinde değişebilir. İçsel bilgiler değişmez ve ortak kullanılabilir; final ya da static düşünülebilecek yerler bunlardır. Bu kalıpta belirli bir nesne için gereksiz kalan bir alan yoktur. Böyle bir durum Composite kalıbında görülmüştü: yaprak düğümlerin çocuğu olamayacağı için bileşen ekleme-çıkarma metotlarının ve alt bileşen listesinin anlamı yoktur. Ama Flyweight'te öyle bir durum yoktur.

### 6.8 Çözüm 2 ile çözüm 3'ün karşılaştırılması

Hocaya göre bellek tasarrufu, esneklik ve senaryo destekleme açısından ikinci ve üçüncü çözüm hemen hemen aynıdır; hangisi kullanılsa çok fark etmez. Yine de istemcinin elini biraz daha rahatlattığı için üçüncü çözüm önerilebilir. Hoca çözüm 2'nin ana programını da 15-10 ve 14-11 gramajlarıyla düzeltip çalıştırdı; "bir şey düzeltelim deyince tek yerden düzeltmek yetmiyor, kaç yerden düzeltmek gerekiyor" diye ekledi.

Aradan sonra hoca, ders notlarında atom ağırlıklarını düzelttiğini ve notların güncellenmiş halini paylaşmak istediğini söyledi.

## 7. Proxy Kalıbı

### 7.1 Amaç ve örnekler

Proxy "vekil" demektir. İnternete eskiden çevirmeli ağlarla bağlanılırken hem yavaşlık hem veri tasarrufu nedeniyle bazı veriler ön bellekten getirilirdi; farklı internet sağlayıcıları farklı proxy'ler sunardı. Aynı mantık bu kalıpta da geçerlidir.

Proxy kalıbının amacı, bir nesneye erişimi denetlemek için bu nesnenin yerine geçecek başka bir nesneyi araya koymaktır.

Örnekler: eski, düşük konfigürasyonlu bilgisayarlarda Word belgesine büyük ve karmaşık çizimler, denklemler, resimler konduğunda sayfalar arasında hızla kaydırırken resim görünmez, onun yerine bir işaret görünürdü; sayfada durulunca ilgili nesne yüklenip net görünürdü. Ağır web sayfalarında yüklenecek resimlerin yerine önce bir çarpı işareti gelirdi. Bugün de bant genişliği arttıkça uygulamaların tüketimi de artıyor: sayfa ana şablonuyla yükleniyor, ayrıntılı resimlerin geleceği yerlerde yükleme simgeleri çıkıyor. Proxy bu gibi senaryolarda kullanılır.

GoF örneğinde metin belgesine bir resim doğrudan değil, bir vekil aracılığıyla gömülür. Grafik yapısı zaten kuruludur; çizimi uzun sürecek grafikler için grafiğin kendisi yerine bir ImageProxy konur. ImageProxy, Graphic soyut sınıfıyla aynı arayüze uyar, ondan kalıtımla türetilmiştir. Resim yüklenmemişse örneğin boyut bilgisi için varsayılan bir değer döndürür, yüklenmişse ilişkili olduğu resmin bilgisini gönderir.

### 7.2 Kalıp yapısı

İstemciler gerçek nesnelerle (RealSubject) doğrudan iletişimde olmaz; onları simgeleyen bir arayüz (Subject) üzerinden konuşur. Bu arayüzle uyumlu vekiller vardır ve bir vekil bir gerçek nesneyle ilişkilidir. Bütün istekler vekile gider, vekil de isteği gerçek nesneye iletir. Gerçek nesnenin yüklenmesi gibi işlerden de vekil sorumludur. İstemci gerçek nesneye ulaşmaz, vekille konuşur; vekil bütün işleri gerçek nesneye yaptırır, ama nesne yüklenmemişse vb. durumlarda araya girerek ek davranış sergiler.

> **Hocanın vurgusu:** Proxy'yi tek kademeli bir Decorator gibi düşünebilirsiniz. Ama esas amacını unutmayalım: yeni bir yetenek vermekten ziyade esas nesneye erişimi denetlemek (02:16:51).

### 7.3 Uygulanabileceği anlar

- Bir nesneye erişimin basit bir işaretçiden fazlasıyla yapılması gereken her durumda. Gereken ek yetenekler vekile kodlanır.
- Kullanılacak nesnenin farklı bir adreste bulunması: ağdaki başka bir bilgisayarın, başka bir Java sanal makinesinin belleğinde çalışan nesnelere erişmek. Java'da bunun için Remote Method Invocation (RMI) altyapısı vardır; arada nesnelerin kaydettirildiği kayıtlar (registry) olur, hoca ayrıntısına girmedi.
- Kelime işlemcilerdeki resim ve başka içerik davranışları gibi durumlar. Hocaya göre bu çözümlerin hepsi Proxy mantığına dayanır.

  > **Ek bilgi (derste anlatılmadı):** GoF, vekilleri amaçlarına göre sınıflar: uzak nesneyi yerelde temsil eden remote proxy, pahalı nesneyi gerektiğinde oluşturan virtual proxy (resim örneği) ve erişim haklarını denetleyen protection proxy (bir sonraki örnek). Akıllı referans (smart reference) da başka bir türdür.

## 8. Örnek Gerçekleme: İzlediğin Kadar Öde Sunucusu

### 8.1 Problem

Bir içerik sağlayıcısının elinde çok sayıda içerik var ve bunu bir sunucu üzerinden internete açabiliyor. Ama sunucuda para tahsilatı, abonelik denetimi gibi bir yetenek yok; böyle açılırsa zarar edilir. İçerikten para kazanmak için pay-per-view, yani izlediğin kadar öde sistemi kurulmak isteniyor. Slayttaki problem (ekrandan, 02:20:00):

- İzlediğin-kadar-öde sunucusuna gelen taleplerin emniyet altına alınması.
- Sunucu kendisine verilen adrese istenen programı gönderecek.
- Ancak göndermeden önce istemcinin ödeme yapıp yapmadığına bakılmalı.
- Sunucu yazılımında bu yetenek yok ve yazılımın kaynak koduna erişilemiyor; elde yalnız binary'leri var.
- Araya konacak bir vekil bu işi çözer (protection proxy).

Vekilin amacı erişimi kontrol altına almak olduğu için literatürde buna protection proxy denir. Hoca örneği, Netflix, Amazon gibi platformlar yokken uydu yayınlarında belirli bir programı ayrıca satın alma sistemine benzetti.

### 8.2 Tasarım

Slayttaki çözüm (ekrandan, 02:22:30):

```
«interface» StreamSource
  + stream(programID, destination : String) : boolean

StreamServer  (StreamSource'u gerçekler; kaynak kodu kayıp olan sunucu)
  + stream(programID, destination : String) : boolean

StreamProxy  (StreamSource'u gerçekler)
  - original : StreamSource            (1 --> 1 StreamSource)
  + StreamProxy(original : StreamSource)
  + stream(programID, destination : String) : boolean
  + checkPayment(programID, destination : String) : boolean

StreamDestination                      (1 --> 1 StreamSource)
  - server : StreamSource
  - address : String
  + StreamDestination( address : String )
  + getAddress( ) : String
  + setServer( StreamSource )
  + makePayment( double ) : boolean
  + requestProgram( programID : String ) : boolean

MainApp --> StreamDestination, StreamServer, StreamProxy
```

Hoca şemada mevcut kısmı (StreamDestination, StreamSource, StreamServer) çizerek ayırdı ve Proxy kalıbına uyarak eklenenin StreamProxy olduğunu gösterdi. İstemcinin asıl bağımlılık noktası StreamSource arayüzüdür. StreamDestination temel yönlendirme uygulaması, yani bir kanal gibi düşünülebilir.

StreamProxy işi tek başına yapmaz; StreamSource türünden bir üye alan aracılığıyla gerçek sunucuyla (StreamServer) ilişkilendirilir. "Şu programı şu adrese gönder" dendiğinde önce checkPayment ile o program için hedefin ödeme yapıp yapmadığına bakar; yapılmışsa isteği gerçek sunucuya iletir ve akış yapılıp true döner, yapılmamışsa akış gönderilmez ve false döner.

### 8.3 Kod

Kodlar iskelet koddur. StreamServer'ın gerçek akışı nasıl yapacağı kodlanmamış, doğrudan true döndürür. StreamDestination'da da adres ve başarılı dönen ödeme metodu gibi iskeletler vardır. StreamProxy'nin kodu (ekrandan, 02:26:40):

```java
public StreamProxy(StreamSource original) {
    this.original = original;
}
public boolean stream(String programID, String destination) {
    if( checkPayment(programID, destination) )
        return original.stream(programID, destination);
    else
        return false;
}
public boolean checkPayment(String programID, String destination) {
    // Ödemeyi destination parametresine göre kontrol et
    // Ödeme yapılmışsa true, aksi halde false döndür.
    return false;
}
```

Hoca, vekilin üyesini StreamServer türünden değil StreamSource arayüzü türünden tanımladığını vurguladı: aynı arayüze uyan başka marka sunucular da olabilir ve bu çözüm onları da destekler. Ortak arayüz olmasaydı farklı sunucular için farklı vekiller gerekecekti. Ana programda önce gerçek sunucu, sonra onu alan vekil oluşturulur; istemciye (StreamDestination)

gerçek sunucu değil vekil verilir ve istemci programı gönder dediğinde vekil üzerinden izleyebilir.

## 9. Chain of Responsibility Kalıbı

Proxy ile yapısal kalıplar biter ve davranışsal (behavioral) kalıplara geçilir. İlk davranışsal kalıp Chain of Responsibility'dir. Hoca, alternatif çözümü olmayan tek çözümlü bir örneği olduğu için derste vaktin yeteceğini söyledi.

### 9.1 Amaç

Slayttaki amaç (ekrandan, 02:30:00):

- Birbirleri ile ilişkili bir nesneler zinciri veya hiyerarşisindeki bir nesnenin, aldığı bir mesajı işleyemeyeceği durumlarda, bu mesajı gerekli işlemi yapabilecek başka bir nesneye iletmek.
- Mesajın işlenememesinin nedeni nesnenin arayüzünde o mesajın olmaması değil, nesnenin mesajı işleyecek yetki veya bilgiye sahip olmamasıdır.

Hoca ikinci maddeyi açtı: nesnede o metot olmasaydı çoğu dilde ona o metot çağrısı en baştan yönlendirilemezdi. Yalnız Smalltalk'ta ilginç bir mekanizma vardır: bir nesnenin sınıfında tanımlanmamış bir metodu çağrılmaya çalışılınca, en üstteki Object sınıfında bulunan doesNotUnderstand metodu çalışır; bu metodun gövdesine kod yazılırsa olmayan metot çağrıldığında o kod yürütülür. C#'ta, C++'ta böyle bir şey yoktur. Burada mesele, metodun imzası tanımlı olduğu halde tam yürütülemeyen bir isteğin onu daha iyi, daha ayrıntılı yürütebilecek bir nesneye iletilmesidir.

### 9.2 Örnek: bağlama duyarlı yardım

Hoca örneği PowerPoint'in paragraf iletişim kutusu üzerinden anlattı. İletişim kutusundaki soru işareti yardım düğmesi genel yardımı açar, ama kullanıcı yalnız bir ayrıntıyı, örneğin "satır aralığı"nın ne demek olduğunu öğrenmek ister. Eski bir Office sürümünde soru işaretine tıklayınca yardım hemen açılmaz, fare imleci soru işaretli bir oka dönüşürdü; hangi öğeye tıklanırsa o öğeyle ilgili en özelleşmiş yardım gelirdi.

Satır aralığı öğesine tıklandığında, o kadar ayrıntılı bir yardım hazırlanmamışsa hiç olmazsa öğenin yer aldığı "aralık" grubunun (önce, sonra, satır aralığı değerleri) yardım sayfası gelsin. O da yoksa bütün iletişim kutusunun içeriğini anlatan yardım gelsin. En azından "Word'e, PowerPoint'e hoş geldiniz" gibi genel bir sayfa çıkmasın.

Slayttaki örnek maddeleri:

- Kullanıcı bir dialog kutusundaki bileşenlerden birinin işlevi hakkında yardım almak istiyor.
- Eğer o bileşene özel bir yardım içeriği varsa gösterilir, aksi halde yardım mesajı o bileşeni içeren bir üst bileşene iletilir.
- İletim, mesaj bir nesne tarafından cevaplanana kadar veya zincirin sonuna gelinceye dek sürer.
- Bir bileşen, hangi üst bileşen içinde yer aldığını bilecektir.

Nesne olarak bakıldığında OK ve Print düğmeleri PrintDialog'un içindedir, PrintDialog da uygulamanın içindedir. OK düğmesine yardım isteği gelir; ilişkili yardım dosyası varsa gösterilir, yoksa içinde bulunduğu PrintDialog'a iletilir; o da gösteremezse uygulamaya iletilir ve uygulama ana yardımı gösterir. Bunun için zincirdeki bütün nesnelerin ortak bir arayüze sahip olması gerekir; yoksa "ilişkili olduğum nesnenin türü şuysa şu metodunu çağır" gibi kod yazmak gerekirdi. Hepsinde örneğin bir HandleHelp metodu vardır; nesne tam yardım edemiyorsa ilişkili olduğu nesnenin aynı metodunu çağırır. Bu örnekte iletişim kutuları, düğmeler, uygulama ve ortak üst sınıf zaten çerçevede vardır ve aralarındaki ilişki zaten kuruludur; yapılan tek şey metot gövdelerini "işini yapabiliyorsan yap, yapamıyorsan ilişkili olduğun nesneye gönder" biçiminde doldurmaktır.

### 9.3 Kalıp yapısı

İsteği ele alacak metodu tanımlayan bir Handler vardır; birden fazla istek türü olabilir. Handler bir arayüz olamaz, çünkü üye alanı vardır: successor, yani kendisini izleyen, içinde bulunduğu nesne. Somut handler'lar Handler'dan kalıtımla türetilir; hepsi aynı soyut sınıfın alt sınıfları olduğu için herhangi bir türden somut alt sınıfla successor ilişkisi kurulabilir. Nesneler bellekte bir zincir oluşturur.

İlgili bileşenler zaten böyle bir hiyerarşi içindeyse kalıbı kurmak kolaydır; değilse hiyerarşiyi de tasarımcı düşünmelidir. Birden fazla istek olabilecekse Handler sınıfında ne kadar farklı istek varsa onların imzaları tanımlanır.

### 9.4 Uygulanabileceği anlar ve dikkat edilecekler

- Bir metodu yürütebilecek, bir mesajı yanıtlayabilecek birden fazla nesne bulunuyorsa.
- Yanıtlayacak nesne önceden bilinemiyorsa.

  **Hocanın vurgusu:** Zinciri doğru yapılandırmalıyız. Aksi halde bir mesajın yanıtının geri döneceğinin garantisi yoktur. Ara düğümlerde olmasa bile hiyerarşinin kökünde mesajın mutlaka ele alınması lazım; yoksa mesaj yanıtsız kalır, kaybolur (02:37:10).

PrintDialog'un ilişkili olduğu bir uygulama nesnesi yoksa, yani zincir doğru sonlandırılmadıysa, iletişim kutusu belki hiç gösterilemez bile; ama burada kalıba yalnız tasarım bakış açısından bakılıyor.

## 10. Örnek Gerçekleme: Reklam Yönetimi

### 10.1 Problem

Bir reklam yönetim firmasıyız. Portföyümüzde gösterilebilecek reklamlar ve bu reklamların gösterilebileceği web siteleri var. En doğru reklamı en doğru demografik gruba, en doğru sitelerde göstermek istiyoruz; buna göre kampanya kuralları hazırlıyoruz. Slayttaki problem (ekrandan, 02:40:00; slaytta kaynak olarak hocanın baş harfleri yazılı, hoca örneği kendisinin oluşturduğunu söyledi):

- Kampanya 1: Spor sitesinin 18-39 yaş arası erkek ziyaretçilerine traş bıçağı reklamı göster, 18-39 yaş arası kadın ziyaretçilerine fitness salonu reklamı göster, diğer ziyaretçilere spor mağazasındaki indirimin reklamını göster.
- Kampanya 2: Haber sitesinin 18-29 yaş arası A gelir grubu erkek ziyaretçilerine otomobil dergisi reklamını göster, aksi halde spor sitesinin kampanyasını aynen uygula.
- Kampanya 3: Ziyaretçinin coğrafi bölgesi ve gelir düzeyine göre portföydeki lokanta reklamlarından birini göster.

Hoca, Kampanya 2'nin bu biçimde verilmesinin alt zincirlerin yeniden kullanılabildiğini göstermek için olduğunu söyledi. Kampanya 3 öbürlerinden bağımsızdır ve tek başına kodlanabilir; örnekte Kampanya 1 ve 2 gerçeklenmiştir.

### 10.2 Tasarım

Paket `dp.chainOfResponsibility.example`:

- User: cinsiyet, gelir düzeyi, yaş; kurucu ve get metotları. Ek olarak yaşın belirli bir aralığa dahil olup olmadığını sınayıp true döndüren bir metot.
- Cinsiyet enum'ı: KADIN, ERKEK. GelirDuzeyi enum'ı: A, B, C. Hoca ilkel türler yerine enum kullandığını belirtti.
- Banner: reklamın gösterileceği afiş (pankart); ekran bitmap'i çizilmez, basit tutulmuştur.
- Reklam: soyut sınıf. Gösterilecek reklamın banner'ı ve bu reklam uygun değilse gösterilecek sıradaki reklamı tutan `sonraki` üyesi. Zinciri kuran şey sonraki üyesidir; o olmasa Chain of Responsibility olmaz.
- Somut reklamlar: TrasBicak, FitnessSalonu, IndirimSporMagaza, OtoDergi.

Reklam'ın iki soyut metodu vardır: reklamGoster(User) ve reklamUygunMu(User). İkincisi reklam kullanıcıya uygunsa true, değilse false döndürür. Her reklamın uygunluğu farklı kurallara

bağlı olduğu için soyut bırakılır. Zinciri işleten metot ise somut reklamKotar(User) metodudur; Chain of Responsibility'nin gerektirdiği çalışma biçimi buna kodlanmıştır.

Reklam sınıfının ilgili kısmı (ekrandan, 02:45:50):

```
public Reklam(Banner banner, Reklam sonraki) {
    this.banner = banner; this.sonraki = sonraki;
}
public abstract boolean reklamUygunMu ( User u );
public abstract void reklamGoster( User u );
public final void reklamKotar( User u ) {
    if( reklamUygunMu(u) ) {
        System.out.println( this.getClass().getName()
                + "->" + u );
        reklamGoster(u);
    }
    else if( sonraki != null ) {
        System.out.println( this.getClass().getName()
                + "->" + sonraki.getClass().getName());
        sonraki.reklamKotar(u);
    }
}
```

> **Hocanın vurgusu:** reklamKotar metodunu özellikle final yaptım. Zincirin idaresi ve iletimi burada kodlu; somut reklam sınıflarında bu çalışma mantığının override edilmesini istemem, bütün işi bozar. Final işaretlendiği için alt sınıflar müdahale edemez (02:45:15).

Reklam uygunsa reklamKotar sınıf adını ve kullanıcıyı yazar ve reklamı gösterir. Uygun değilse ve sonraki bir reklam varsa "bunu işleyemedim, şuna iletiyorum" anlamında kendi adını ve sonrakinin adını yazar, isteği sonrakine iletir. Kampanya kuralları doğru yapılandırıldıysa her kişiye mutlaka bir reklam gösterilir; birine hiç reklam gösterilemiyorsa kural zinciri doğru tanımlanmamış demektir. Hoca, GoF örneğinde hazır bir ilişki hiyerarşisi (her bileşen bir uygulamanın parçası) olduğunu, burada ise zinciri kendisi kurduğu için doğru yapılandırmaya ek özen göstermesi gerektiğini söyledi.

Somut reklamların uygunluk kuralları:

- TrasBicak.reklamUygunMu: kullanıcı erkek ve yaşı 18-39 aralığındaysa true.
- FitnessSalonu.reklamUygunMu: kullanıcı kadın ve yaşı 18-39 aralığındaysa true.
- IndirimSporMagaza.reklamUygunMu: hiçbir koşul olmadan doğrudan true; bu reklam herkese uygundur.

### 10.3 Ana program ve Kampanya 1

Dört kullanıcı tanımlanır (ekrandan, 02:53:00):

```
users = new User[4];
users[0] = new User( Cinsiyet.ERKEK, GelirDuzeyi.C, 23 );
users[1] = new User( Cinsiyet.ERKEK, GelirDuzeyi.A, 53 );
users[2] = new User( Cinsiyet.KADIN, GelirDuzeyi.B, 33 );
users[3] = new User( Cinsiyet.ERKEK, GelirDuzeyi.A, 28 );
```

Kampanya 1'in reklamları bir diziye konur (ekrandan, 02:50:00):

```
public void kampanya1( ) {
    Reklam[] reklamlar = new Reklam[3];
    reklamlar[2] = new IndirimSporMagaza(new Banner(), null);
    reklamlar[1] = new FitnessSalonu(new Banner(), reklamlar[2]);
    reklamlar[0] = new TrasBicak(new Banner(), reklamlar[1]);
    reklamlar[0].reklamKotar(users[0]);
    reklamlar[0].reklamKotar(users[1]);
    reklamlar[0].reklamKotar(users[2]);
}
```

Reklamlar birbirine bağımlı olduğu için (her biri sonrakini kurucusunda alır) dizi sondan başa doğru kurulur: önce 2, sonra 1, sonra 0. Zincirin başında tıraş bıçağı, sonra fitness salonu, en sonda herkese uygun spor mağazası indirimi vardır; IndirimSporMagaza'nın sonrakisi null'dır. İstek her zaman ilk reklamdan başlatılır. Hoca, bütün akışı göstermek için üç kullanıcının bu kampanyada yettiğini söyledi.

Kampanya 1'in çıktısı (ekrandan, 02:53:00):

```
dp.chainOfResponsibility.example.TrasBicak->Erkek,C,23
dp.chainOfResponsibility.example.TrasBicak->dp.chainOfResponsibility.example.FitnessSalonu
dp.chainOfResponsibility.example.FitnessSalonu->dp.chainOfResponsibility.example.IndirimSporMa
dp.chainOfResponsibility.example.IndirimSporMagaza->Erkek,A,53
dp.chainOfResponsibility.example.TrasBicak->dp.chainOfResponsibility.example.FitnessSalonu
dp.chainOfResponsibility.example.FitnessSalonu->Kadın,B,33
```

Adım adım:

- 23 yaşındaki erkek: tıraş bıçağı reklamı 18-39 yaş arası erkeklere uygundur; reklam doğrudan gösterilir.
- 53 yaşındaki erkek: erkek olsa da yaşı aralıkta değildir, tıraş bıçağı uygun değildir ve fitness salonuna iletir. Fitness salonu kadın kullanıcılar içindir, cinsiyet uymaz ve indirime iletir. İndirim reklamı herkese uygundur ve gösterilir.
- 33 yaşındaki kadın: tıraş bıçağı uymaz, fitness salonuna iletilir; kadın ve 18-39 yaş aralığında olduğu için fitness salonu reklamı gösterilir.

### 10.4 Kampanya 2

Kampanya 2'de dört elemanlı bir dizi kurulur; indirim (3), fitness salonu (2) ve tıraş bıçağı (1) Kampanya 1'deki gibi zincirlenir, en başa otomobil dergisi reklamı konur (ekrandan, 02:50:00; dizinin ilk elemanının kuruluşu ekranda görünmüyor, anlatımdan). Kural: 18-29 yaş arası, A gelir grubu erkeklere otomobil dergisi; aksi halde zincirin geri kalanı aynen uygulanır.

- 23 yaşındaki erkek genç ve erkektir ama gelir grubu C olduğu için otomobil dergisi reklamı ona gösterilmez; istek tıraş bıçağına iletilir ve tıraş bıçağı reklamı gösterilir.
- 53 yaşındaki erkek yaşı nedeniyle otomobil dergisini görmez; zincir Kampanya 1'deki gibi indirime kadar iner.
- 28 yaşındaki A gelir grubu erkek otomobil dergisi reklamını görür.

Hoca, zincirin doğru yapılandırıldığını ve herkese bir reklam gösterilebildiğini belirterek örneği bitirdi.

## Örnek Sorular

**Soru 1** (ekrandan, 00:10:00; derste sözlü, 00:11:22): TextArea'dan türeyen ScrollTextArea ve BorderTextArea draw() metodunu override ediyor; ikisinden çoklu kalıtımla türeyen ScrollBorderTextArea draw()'u override etmiyor. Bu sınıfın bir nesnesinde draw() çağrılırsa hangi metot çalışır? Bu tasarımın başka zayıf yönleri nelerdir?

**Cevap:** Hocanın cevabı: bu diamond kalıtım probleminin belirsizliğidir. Dil, kalıtım listesindeki ilk ya da son ebeveyninkini geçerli sayabilir ya da (C++'ta anlatıldığı gibi) böyle metotların en alt sınıfta mutlaka override edilmesini zorunlu tutup aksi halde derleme hatası verebilir. Zayıf yönler (slayttan): Scroll ve Border yetenekleri iki ayrı yerde tekrar gerçeklenir; Panel'e de aynı yetenekleri kazandırmak için üç sınıf daha gerekir; ayrıca çözüm yalnız çeşitlilik sağlar, var olan bir nesne çalışma anında yeniden yapılandırılamaz.

**Soru 2** (ekrandan, 00:50:00; derste sözlü, 00:49:22–00:51:30): Header'lar önce kendini basıp sonra `super.prtTicket()` çağırıyor, Footer'lar önce `super.prtTicket()` çağırıp sonra kendini basıyor. Bileşen sırasıyla SalesTicket, Footer1, Footer2, Header2, Header1 ile sarılırsa çıktı ne olur? Footer'lar header'lardan sonra ilişkilendirilseydi ne değişirdi?

**Cevap:** Derste çalıştırılarak gösterildi: Header1, Header2, SalesTicket, Footer1, Footer2. En dıştaki Header1 önce kendini basar; en içteki footer (Footer1) içteki işi yaptırdıktan hemen sonra, yani önce basılır. Footer'lar header'lardan sonra ilişkilendirilse de footer'lar her zaman makbuzdan sonra, header'lar her zaman önce basılır; yalnız aynı türden dekoratörlerin

kendi aralarındaki sıra, ilişkilendirme sırasına göre değişir. (Üç farklı sarma sırası python ile denetlendi.)

**Soru 3** (ekrandan, 02:00:00; derste sözlü, 01:35:18–01:44:21): Kimya örneğinde karbon için tek ortak bir KimyasalMadde nesnesi kullanılıyor (çözüm b). Kara barutta karbon 15 gram ayarlanıp eklendikten sonra temiz barut için `c.setGram(14)` yapılıp eklenirse tarifler ne gösterir? Neden?

**Cevap:** Derste çalıştırılarak gösterildi: iki karışımda da "14.0gr Karbon" görünür. Gram bilgisi paylaşılan madde nesnesinin içinde tutulduğu için son atama bütün karışımları etkiler. Ayrı nesneler kullanan çözüm a doğru sonuç verir ama bellekte tekrar yaratır. Doğru çözüm gramı dışsal bilgi olarak Bileşen sınıfına taşımak ve madde nesnelerini MaddeFabrikasi üzerinden paylaşmaktır (Flyweight).

**Soru 4** (ekrandan, 02:40:00, 02:53:00; derste sözlü, 02:50:08): Kampanya 2 zinciri OtoDergi → TrasBicak → FitnessSalonu → IndirimSporMagaza olarak kuruluyor. Kullanıcılar (ERKEK, C, 23), (ERKEK, A, 53), (KADIN, B, 33), (ERKEK, A, 28) için hangi reklamlar gösterilir? Reklam gösterilmeyen kişi olur mu?

**Cevap:** Derste hoca birinci, ikinci ve dördüncü kullanıcıyı sözlü olarak izledi. Kurallar uygulandığında: (ERKEK, C, 23) gelir grubu nedeniyle dergiyi görmez, tıraş bıçağı reklamını görür. (ERKEK, A, 53) yaşı nedeniyle dergiyi ve tıraş bıçağını, cinsiyeti nedeniyle fitness salonunu görmez; spor mağazası indirimini görür. (KADIN, B, 33) dergiyi ve tıraş bıçağını görmez, fitness salonu reklamını görür. (ERKEK, A, 28) otomobil dergisi reklamını görür. Zincirin sonunda herkese uygun indirim reklamı olduğu için reklam gösterilmeyen kimse olmaz. (Zincir python ile denetlendi.)

**Soru 5** (derste sözlü, 01:53:36): Flyweight örneğinde her nesne için gerekmeyen ya da kullanılmayan alanların olması bir olumsuzluk yaratmaz mı?

**Cevap:** Hocanın cevabı: bu kalıpta belirli bir nesne için gereksiz kalan bir alan yoktur. Gram, aynı madde farklı bileşenlerde farklı miktarlarda kullanıldığı için dışsal bilgidir ve zamanla değişebilir; final ya da static düşünülebilecek olanlar içsel bilgilerdir. Gereksiz alan durumu Composite kalıbında görülür: yaprak düğümlerde bileşen ekleme-çıkarma metotlarının ve alt bileşen listesinin anlamı yoktur.

## Dersin Toparlaması

Bu derste yapısal kalıplardan Decorator, Facade, Flyweight ve Proxy tamamlandı ve davranışsal kalıplara Chain of Responsibility ile geçildi. Decorator, kalıtımın kodlama zamanında ve bütün nesneler için yaptığı özelleştirme yerine nesneleri birbirine bağlayarak çalışma anında sorumluluk eklemeyi gösterdi; makbuz örneğinde dekoratörlerin önce ya da sonra çalışmasının çıktı sırasını nasıl belirlediği görüldü. Facade karmaşık bir alt sisteme tek erişim noktası sağlar. Flyweight, durumu içsel ve dışsal olarak ayırıp çok sayıda nesneyi paylaşılan az sayıda nesneyle temsil eder; kimya örneğinde "neyin değiştiğini bul ve onu sarmala" ilkesiyle gram bilgisinin ayrılması kalıba doğal olarak götürdü. Proxy, aynı arayüze uyan bir vekille gerçek nesneye erişimi denetler ve tek kademeli bir Decorator'a benzer ama amacı farklıdır. Chain of Responsibility'de bir isteği işleyemeyen nesne onu zincirdeki bir sonrakine iletir; zincirin doğru kurulması ve zinciri yöneten metodun alt sınıflarca bozulamaması önemlidir.

Hoca, davranışsal kalıplara bir sonraki derste Command kalıbıyla devam edileceğini söyledi.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.